

What We Store in Redis — HRMS

Plain-language guide · QuikIT HRMS · 11 June 2026

Names like `auth:session` are just **namespaces** — Redis has no real folders, the browser splits keys on the `:` character. So `auth:session:87f1a7e...` is one single key shown as *auth* › *session*. Everything here falls into **two buckets**: a short-lived **cache** (fast copies of slow data) and a **job queue** (background to-do list).

The keys, one by one

The login session `auth:session`

What The central SSO login session (the `87f1a7e-...` is your session id). ~184 bytes.
When Written when you log in.
Why So every app (HRMS, QuikScale, etc.) shares one login. TTL is **29 days** = how long you stay signed in.

"Is this login still good?" `session-valid:{userId}`

What A tiny yes/no flag per user.
When Set on the first API request, refreshed roughly every 60 seconds.
Why Otherwise every HRMS request would call the central auth server to ask "is this user still valid / not logged out?" — slow. We cache "valid" for up to 60s. On logout the key is deleted so the next request re-checks instantly.

Your permissions `perms:{tenantId}:{userId}`

What What you're allowed to do (admin vs employee, which actions).
When Computed once per user, reused on every request.
Why Resolving roles hits the database. Cache it so the permission check on each page/API is instant. Busted on logout or when your role changes.

Your profile `employee-me:{tenantId}:{userId}`

What Your employee record (name, department — the "me" data the UI shows).
When Loaded when you open the app / your profile.
Why The header, sidebar and many screens need "who am I" on every load. Caching avoids re-reading your row from Postgres each time. Busted when your profile is edited.



Birthdays widget

`birthdays:{tenantId}`

- What** List of employees with upcoming birthdays for the dashboard. ~104 bytes.
- When** First time the dashboard / birthday widget loads.
- Why** Birthdays barely change, so it's cached for **5 minutes**. Saves scanning all employees on every dashboard open. Shared per tenant, not per user.



Central membership

`central-member:{tenantId}:{userId}`

- What** Result of the central member-lookup (links the HRMS employee to the central QuikIT account).
- When** Checked during the central-SSO sync flow.
- Why** Avoids calling the central app's lookup endpoint on every request. Part of the central-SSO migration work.



Background jobs

`bull` *(not a cache)*

- What** BullMQ. The 62 keys / 91% is just the queue's internal bookkeeping (jobs, their state, retries).
- When** Whenever a background job is queued — sending emails (offer letters, invites), notifications, etc.
- Why** Heavy/slow work (email, AI document parsing) shouldn't block the web request, so it's pushed to a queue and a worker processes it. Job state lives here in Redis.

Quick reference table

Key	Bucket	Lifetime	Cleared when
<code>auth:session</code>	Login token	29 days	Logout / expiry
<code>session-valid:{userId}</code>	Cache	~60 sec	Logout
<code>perms:{tenant}:{user}</code>	Cache	Short TTL	Logout / role change
<code>employee-me:{tenant}:{user}</code>	Cache	Short TTL	Logout / profile edit
<code>birthdays:{tenant}</code>	Cache	5 min	TTL expiry
<code>central-member:{tenant}:{user}</code>	Cache	Short TTL	Logout / TTL expiry
<code>bull</code>	Job queue	Per job	Job completes

The simple summary

There are really **two different uses of Redis** here:

1. **A cache** (`session-valid` , `perms` , `employee-me` , `birthdays` , `central-member`) — copies of data that's expensive to fetch, kept for a short time so the app is fast and doesn't hammer Postgres / central-auth on every click. All have a TTL and get **deleted on logout or when the underlying data changes**.

2. **A job queue** (`bull`) — the to-do list for background work (emails, notifications), processed by a worker.
And `auth:session` is the actual login token shared across all QuikIT apps.

Scope note: the HRMS Redis footprint was intentionally **trimmed** — list/config caches removed, job-state moved to Postgres, logout busts the auth caches. What appears in the Redis browser matches that trimmed set exactly (only hot-path per-user caches + the birthday widget + `bull`), so nothing here is stray.